

Active Learning applied on text classification

Malin Mattiasson

Master's Thesis in Statistics
Stockholm, Sweden
Spring 2023

Department of Statistics
Stockholm University

Supervisor:
Johan Koskinen

No. 2023:01



Abstract

In many machine learning applications, especially in text classification, gathering and labeling data is a time-consuming and costly task. Therefore, it is desirable to minimize the amount of data that has to be labeled. One way to do so is to apply active learning, which iteratively selects instances to be labeled.

In this thesis, the effect of applying active learning, and more specifically uncertainty sampling, in a text classification setting is examined. The thesis examines the performance of three different uncertainty sampling strategies, namely least confident sampling, margin sampling, and entropy sampling, and whether the classification model has an impact on their performance. The performance is compared to random sampling and is evaluated using two different text datasets, both containing four possible labels

The results showed that margin sampling, under a fixed labeling budget of 2000 labeled instances, consistently outperformed random sampling and the other uncertainty sampling strategies. It was also found that the classification model had an impact on the performance of the query strategies, especially on least confident sampling and entropy sampling. Lastly, it was found that applying uncertainty sampling using only a fraction of the training set could produce a model with similar, and sometimes even better, performance compared to when using the full training set.

Acknowledgements

First and foremost, I would like to thank my supervisor Johan Koskinen for his support and valuable feedback throughout the thesis. I would also like to thank Frank Miller for his valuable comments during the early stages of my thesis. Lastly, I would like to express my gratitude to my family and my friends for believing in me and supporting me during my studies. These last two years would not have been possible without you.

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Objective	1
1.3	Delimitations	2
1.4	Ethical Considerations	2
1.5	Outline	2
2	Active Learning	3
2.1	Scenarios	4
2.1.1	Membership Query Synthesis	4
2.1.2	Stream-Based Selective Sampling	4
2.1.3	Pool-Based Active Learning	4
2.2	Query Strategies	5
2.2.1	Random Sampling	5
2.2.2	Uncertainty Sampling	5
2.2.2.1	Least Confident Sampling	6
2.2.2.2	Margin Sampling	6
2.2.2.3	Entropy Sampling	7
2.2.2.4	Previous Studies	8
2.3	Stopping Criterion	8
3	Text Classification	9
3.1	Text Pre-Processing	9
3.2	Feature Representation	10
3.2.1	Bag of Words	10
3.2.2	TF-IDF	11
3.3	Classification Model	12
3.3.1	Logistic Regression	12
3.3.1.1	Parameter Estimation	13
3.3.2	Multinomial Naive Bayes	15
4	Data	17
4.1	Students Questions Dataset	17
4.2	E-commerce Dataset	18
5	Method and Implementation	19
5.1	Split	19
5.2	Pre-Processing	19
5.3	Feature Representation	20
5.4	Classification Models	20
5.5	Performance Metrics	21
5.6	The Active Learning Cycle	22
6	Result	25
6.1	Students Questions Dataset	25
6.2	E-commerce Dataset	26

7	Discussion	28
8	Conclusions	30
	Bibliography	31
	Appendix A Data	33
A.1	Ten Most Common Tokens Students Questions Dataset	33
A.2	English Stop-Words	33
	Appendix B Code	34
B.1	Initial Labeled Set	34
B.2	Query Strategies	34

1 Introduction

1.1 Background

With the increasing number of data-generating devices, there is an abundance of data. The availability of large amounts of data is one of the key factors that has driven the success of *Machine Learning* in recent years. Briefly, machine learning can be explained as teaching computers to learn patterns from data and using those patterns to make decisions or predictions on new, unseen data. Any machine learning model needs to be trained on a large amount of data in order to perform well. In *Supervised Learning*, the data is required to be labeled so that the model can learn to map the features to the associated label. Sometimes these labels are free or very inexpensive, like when you mark e-mails as spam in your inbox or when you rate a movie on a website. However, in many situations, data labeling is performed manually, which can be an expensive and time-consuming task, especially for larger datasets. In these situations, it is desirable to produce models with high performance using fewer labeled examples.

One technique to reduce the amount of labeled data is called *Active Learning*, also referred to as *Experimental Design* in statistics (Kiefer, 1959; Fedorov, 1972). Active learning is an iterative process where the model actively requests the most informative instance to be labeled. This can be compared to the traditional method of randomly choosing which instances to label, namely *Random Sampling*, also referred to as *Passive Learning*. By letting the model choose the data on which it trains, active learning is supposed to produce a model with high accuracy, using fewer labeled instances (Settles, 2010). This means that the cost and time of labeling can be significantly reduced.

A natural field for the application of active learning is text classification. Every day, an enormous amount of text data is generated in organizations and by people, such as social media posts, support requests, and product reviews. In order to make sense of this data, human annotators are needed to categorize it properly, which is a time-consuming and costly task due to its unstructured nature. Applying active learning in these situations can reduce the labeling effort, by only including the most informative instances.

1.2 Objective

The main objective of this thesis is to demonstrate the application of active learning in the context of text classification and examine its impact on a classification model's performance. The thesis seeks to answer the following questions:

- Does active learning, and more specifically uncertainty sampling, produce a better model compared to random sampling under a fixed labeling budget?
- Does the performance differ depending on the query strategy used?
- Does the performance differ depending on the classification model used?

1.3 Delimitations

The thesis is limited to studying the pool-based active learning setting for text classification, in which the classification model has access to a large collection of unlabeled text instances, for two example datasets. The thesis is also limited to studying query strategies based on uncertainty sampling. These include least confident sampling, margin sampling, and entropy sampling, which are introduced in Section 2.2.2.

Since the main goal of this thesis is to explore the impact of applying uncertainty sampling in a text classification setting, and not to find the most accurate classification model, the models used in the thesis are not optimized by testing different hyperparameter settings. Furthermore, the thesis is limited to investigating the performance of only two classification models, namely multinomial logistic regression and multinomial naive Bayes.

The thesis is also limited to applying active learning to two datasets which both contain four possible outcome classes. The choice of including two datasets with the same number of outcome classes is to ensure consistency when evaluating the performance of the query strategies. Lastly, it is assumed that the data is correctly classified. Therefore, this thesis does not account for the possibility of mislabeled instances, which could affect the performance of the models negatively.

1.4 Ethical Considerations

The implementation of active learning could potentially have some negative societal impacts. For example, active learning strategies, if successfully implemented, could reduce the need for human annotators. This can potentially result in job displacement or loss of employment for those who previously performed these tasks.

1.5 Outline

The thesis is divided into eight chapters. In this chapter, an introduction is given followed by delimitations and ethical considerations. Chapter 2 presents the key concepts of active learning. In Chapter 3, the different steps of text classification are presented. These steps include pre-processing, feature representation, and the classification models used in this thesis. Chapter 4 presents the datasets. In Chapter 5, the details of the method and the implementation of the query strategies are presented. In Chapters 6 and 7, the results are presented and discussed. Finally, in Chapter 8, the main conclusions from the thesis are presented.

2 Active Learning

Mitchell (1997, p. 2) defines machine learning as:

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .”

In the machine learning setting, the experience E is data, meaning that the computer program learns from input data. The two most common ways of learning are unsupervised and supervised learning. In unsupervised learning, all data points are unlabeled. The goal is to create a model that can reason about important properties of the data (Lindholm et al., 2022, p. 259). In supervised learning, on the other hand, the model is trained on labeled data. The main goal is to create a computer program that can predict or classify unseen data instances with high accuracy (Lindholm et al., 2022, p. 4).

Settles (2010, pp. 3–4) describe active learning as a sub-field of supervised machine learning where the model is able to choose which instances it wants to be trained on. It can be described as a process consisting of four main components:

- **Unlabeled Data:** An unlabeled dataset, $\mathcal{U} = \{\mathbf{x}_u\}_{u=1}^U$, with no information about the dependent variable y . In a text classification setting, y is the label of the text instance.
- **Machine Learning Model:** The machine learning model $\mathcal{M}$ plays a central role in the active learning process. The chosen model uses a set of labeled data points to train a model and is then applied on $\mathcal{U}$ to estimate the conditional class probabilities for each instance. By using some pre-defined informativeness metric, the most informative instances in $\mathcal{U}$ are queried to be labeled the oracle (see below).
- **Oracle:** The oracle, which is usually a human annotator, provides labels for the data points selected to be most informative. In most active learning studies, the oracle is assumed to accurately label the instances. However, in real-world applications, oracles are often imperfect and can provide inaccurate labels, which can negatively affect the active learning process and the overall performance of the model (Settles, 2010, p. 36). An assumption made in this thesis is that the text instances have been correctly labeled.
- **Labeled Data:** The labeled dataset, $\mathcal{L} = \{\mathbf{x}_l, y_l\}_{l=1}^L$, initially consists of a small amount of labeled data that the model can train on. The instances labeled by the oracle are successively added to $\mathcal{L}$, meaning that the input data for the model becomes larger for every query.

The biggest difference compared to ordinary supervised learning is that active learning does not assume a pre-existing large amount of labeled data. Instead, it starts with a small initial labeled training set, and then actively requests the most informative instances to be labeled by the oracle. The key assumption is that since only the most informative instances are added to the labeled training set $\mathcal{L}$, the model should be able to perform better with less training data (Settles, 2010, p. 3).

2.1 Scenarios

The main process of active learning is to query an oracle to label new instances. Settles (2010) mentions three different scenarios of querying: *Membership Query Synthesis*, *Stream-Based Selective Sampling* and *Pool-Based Active Learning*.

2.1.1 Membership Query Synthesis

In membership query synthesis, the machine learning model typically requests labels for instances generated synthetically. In other words, the model constructs a synthetic instance that would be informative to label. Even though this query scenario may be reasonable in some situations, it can be problematic in for example vision-based tasks (Settles, 2010, p. 9–10). This was the case for Lang & Baum (1992) who employed membership query synthesis to train a classification model to classify handwritten characters. A problem they encountered was that some of the query images generated by the model consisted of unrecognizable symbols, which were impossible to reasonably label.

2.1.2 Stream-Based Selective Sampling

Another scenario is stream-based selective sampling. In this scenario, an instance is typically drawn from the unlabeled data one at a time, and the machine learning model decides, based on the instance’s information content, whether to query the unlabeled instance or discard it. The key assumption is that acquiring an unlabeled instance is free or inexpensive. (Settles, 2010, pp. 10–11)

2.1.3 Pool-Based Active Learning

The third scenario is pool-based active learning, which is suitable in situations where a large amount of unlabeled data instances can be collected at once. It is often visualized as an iterative cycle, as shown in Figure 1. To start with, a small number of instances are labeled and put in the labeled training set, $\mathcal{L}$. The rest of the unlabeled instances are usually denoted as the unlabeled pool, $\mathcal{U}$. The model is initially trained on $\mathcal{L}$ and is then applied on $\mathcal{U}$ to evaluate which instances are the most informative. The most informative instance will be queried for annotation by the oracle. Once the selected instance has been labeled, it is removed from $\mathcal{U}$ and added to $\mathcal{L}$. Then, the cycle is restarted and the model is re-trained with the newly collected data and queries new data from the unlabeled pool. This process continues until a predefined stopping criterion has been met. (Settles, 2010, pp. 11–12)

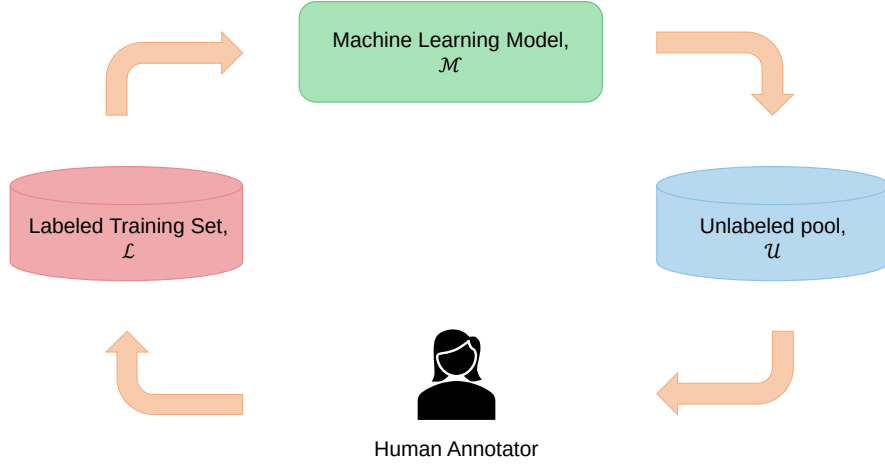


Figure 1: Pool-based active learning (Adapted from Settles, 2010)

2.2 Query Strategies

As mentioned earlier, active learning aims to minimize the amount of training data by only querying the most informative instances. In order to decide which instance is most informative, a query strategy must be determined. An overview of available strategies is given by Settles (2010, pp. 12–21). In this thesis, the approaches of *Random Sampling* and *Uncertainty Sampling* will be considered, which are presented below.

2.2.1 Random Sampling

The simplest way to query observations to the oracle is by using random sampling. With random sampling, instances are randomly selected from the unlabeled pool without replacement. It is common to use the random sampling strategy as a baseline reference (see for example Lewis & Gale, 1994; Tong & Koller, 2001), and this thesis employs the same approach.

2.2.2 Uncertainty Sampling

Uncertainty sampling is an active learning sampling strategy that queries the instance that the classification model is least certain about (Settles, 2010, p. 11). For probabilistic models, i.e. models that output conditional class probabilities, this method is straightforward, since these probabilities can be used directly as a measure of uncertainty for each instance $\mathbf{x}_i$.

As shown in Figure 1, each iteration of the pool-based active learning cycle starts with a labeled training set $\mathcal{L}$. Let $\{1, \dots, k\}$ be the set of all possible labels. If a probabilistic classifier is trained on $\mathcal{L}$ and applied on $\mathcal{U}$, then $P(y_i = k | \mathbf{x}_i, \boldsymbol{\beta})$ indicates the predicted class probability of the i th instance in $\mathcal{U}$. The objective of uncertainty sampling is to, at each iteration, identify the most uncertain instance

$\mathbf{x}^*$ in the unlabeled pool to query the oracle and add to $\mathcal{L}$, based on the predicted class probabilities.

Intuitively, uncertainty sampling selects the instances that lie close to a classification decision boundary. In a binary classification problem, instances with a predicted class probability close to 0.5 are the instances most difficult to classify. However, for multi-class classification problems uncertainty can be quantified in different ways. Settles (2010, pp. 12–13) describes three uncertainty-based strategies that are suitable when there are more than two classes: *Least confident sampling*, *Margin sampling*, and *Entropy sampling*. In the following sections, the three different uncertainty-based strategies along with a demonstration of the computation for each strategy will be presented. The computation will be based on a fictive example consisting of three text instances $\mathbf{x}_1, \mathbf{x}_2$ and $\mathbf{x}_3$, and four class labels 1, 2, 3 and 4. The (fictional) predicted class probabilities for each text instance are presented below:

	1	2	3	4
$\mathbf{x}_1$:	[0.20	0.30	0.10	0.40]
$\mathbf{x}_2$:	[0.15	0.25	0.30	0.30]
$\mathbf{x}_3$:	[0.35	0.25	0.20	0.20]

2.2.2.1 Least Confident Sampling

Least confident sampling is an uncertainty-based strategy that queries the instance that the model is least confident about (Settles, 2010, p. 12). It is defined as follows:

$$\mathbf{x}_{LC}^* = \underset{\mathbf{x}}{\operatorname{argmax}} [1 - P(y_1^* | \mathbf{x}, \beta)],$$

where y_1^* is the most probable label.

The computation of $\mathbf{x}_{LC}^*$ is demonstrated in Figure 2. For the first text instance $\mathbf{x}_1$, the most probable label is $y_1^* = 4$ with a confidence of 0.40. This means that the uncertainty score according to least confident sampling is $1.00 - 0.40 = 0.60$. The uncertainty score for $\mathbf{x}_2$ and $\mathbf{x}_3$ are calculated in the same manner. According to least confident sampling, $\mathbf{x}_2$ is the instance the model is most uncertain about.

	1	2	3	4		
$\mathbf{x}_1$:	[0.20	0.30	0.10	0.40]		[0.60]
$\mathbf{x}_2$:	[0.15	0.25	0.30	0.30]		[0.70]
$\mathbf{x}_3$:	[0.35	0.25	0.20	0.20]		[0.65]

Figure 2: Computation of $\mathbf{x}_{LC}^*$

2.2.2.2 Margin Sampling

Margin sampling is an uncertainty strategy proposed by Scheffer, Decomaine & Wrobel (2001) which queries the instance with the smallest margin between the

predicted conditional class probabilities for the two most probable labels. It can be defined as follows:

$$\mathbf{x}_M^* = \underset{\mathbf{x}}{\operatorname{argmin}} [P(y_1^*|\mathbf{x}, \boldsymbol{\beta}) - P(y_2^*|\mathbf{x}, \boldsymbol{\beta})],$$

where y_1^* and y_2^* are the first and second most probable labels.

In Figure 3, the computation of the $\mathbf{x}_M^*$ is demonstrated. The first and second most probable labels for $\mathbf{x}_1$ are 4 and 2, with a confidence of 0.40 and 0.30, respectively. Hence, $\mathbf{x}_M^* = 0.40 - 0.30 = 0.10$. The uncertainty score for $\mathbf{x}_2$ and $\mathbf{x}_3$ are calculated in the same manner. According to margin sampling, the model is most uncertain about $\mathbf{x}_2$.

	1	2	3	4		
$\mathbf{x}_1$:	[0.20	0.30	0.10	0.40]		[0.10]
$\mathbf{x}_2$:	[0.15	0.25	0.30	0.30]	→	[0.00]
$\mathbf{x}_3$:	[0.35	0.25	0.20	0.20]		[0.10]

Figure 3: Computation of $\mathbf{x}_M^*$

2.2.2.3 Entropy Sampling

Another uncertainty metric is entropy, which was first introduced by Shannon (1948). As opposed to least confident sampling and margin sampling which only considers either the most or the two most probable labels, entropy sampling considers the entire distribution of probabilities across all labels.

Using entropy sampling, the queried instance will be the one with the greatest entropy, which can be described mathematically as:

$$\mathbf{x}_H^* = \underset{\mathbf{x}}{\operatorname{argmax}} \left[- \sum_{k=1}^K P(y = k|\mathbf{x}, \boldsymbol{\beta}) \log P(y = k|\mathbf{x}, \boldsymbol{\beta}) \right],$$

where k ranges over the possible labels.

In Figure 4 the computation of $\mathbf{x}_H^*$ is demonstrated. For $\mathbf{x}_1$, $\mathbf{x}_H^*$ is given by:

$$- [(0.20 * \ln 0.20) + (0.30 * \ln 0.30) + (0.10 * \ln 0.10) + (0.40 * \ln 0.40)] \approx 1.28$$

The uncertainty score for $\mathbf{x}_2$ and $\mathbf{x}_3$ are calculated in the same manner. According to entropy sampling, the model is most uncertain about $\mathbf{x}_3$.

	1	2	3	4		
$\mathbf{x}_1$:	0.20	0.30	0.10	0.40]		[1.28]
$\mathbf{x}_2$:	0.15	0.25	0.30	0.30]		[1.35]
$\mathbf{x}_3$:	0.35	0.25	0.20	0.20]		[1.36]

Figure 4: Computation of $\mathbf{x}_H^*$

2.2.2.4 Previous Studies

There have been several studies applying uncertainty sampling in classification tasks. Lewis & Gale (1994) applied uncertainty sampling on a binary text classification task using logistic regression. They found that uncertainty sampling resulted in a significant reduction in the number of labeled instances needed to achieve high classification accuracy compared to random sampling. Joshi, Porikli & Papanikolopoulos (2009) applied entropy sampling, margin sampling (in their paper referred to as the Best-versus-Second-Best), and random sampling to three image datasets using Support Vector Machines (SVMs). They found that margin sampling significantly reduced the number of labeled instances and that it performed better than both entropy sampling and random sampling. Körner & Wrobel (2006) applied least confident sampling, margin sampling, entropy sampling, specific disagreement (a "control" sampling strategy), and random sampling on twelve different datasets using ensemble-based active learning, which is an active learning strategy that trains multiple models and uses their combined predictions to identify the most informative instances. They found that margin sampling outperformed the other measures in three out of four tested scenarios.

2.3 Stopping Criterion

Since the main goal of active learning is to train a model with high classification accuracy while keeping the cost of labeling low, a stopping criterion is needed to decide when to stop adding new observations to $\mathcal{L}$. According to Settles (2010, p. 44), the stopping criterion can be based on either external or intrinsic factors:

- External factors refer to factors outside of the classification model's control. These factors can for example include the availability and quality of unlabeled data, and the cost and time required to label additional samples.
- Intrinsic factors are factors internal to the classification model. A stopping criteria based on an intrinsic factor could for example be the point where the performance of the model reaches a plateau and further improvement is unlikely to be achieved by labeling additional samples.

Since the aim of this thesis is to examine the impact of uncertainty sampling in comparison to random sampling, the stopping criterion will be based on a predetermined number of labeled instances.

3 Text Classification

From a statistical standpoint, text classification involves predicting the conditional class probabilities:

$$P(y = k|\mathbf{x}, \boldsymbol{\beta}),$$

where $\{1, 2, \dots, K\}$ are the possible output labels, $\mathbf{x}$ is the input text data and $\boldsymbol{\beta}$ are the model parameters.

This chapter starts by providing the theoretical background on how to pre-process and represent text data and ends with introducing the two classification models used in this thesis, namely multinomial logistic regression and multinomial naive Bayes.

3.1 Text Pre-Processing

Text data usually contains a lot of noise such as special characters, repetitive words, and text in different cases, which does not carry much information when it comes to analyzing and classifying text. Therefore, an important step in any text classification task is to pre-process the data. Sarkar (2019, pp. 107–165) presents some common pre-processing steps suitable for text data, some of which, relevant for this thesis, are presented below:

- **Tokenization** Tokenization refers to the procedure of breaking down a text into smaller components, known as tokens, which are usually words or phrases. For English, tokenization is often performed by using punctuation marks and whitespace as delimiters to separate the text into distinct tokens.
- **Noise removal** Noise removal includes filtering out irrelevant and unwanted elements from the data. This can for example include removing special symbols and repeating characters and tokens.
- **Case normalization** Case normalization is the process of converting all text to either upper- or lowercase. This reduces the number of features since it eliminates the duplication of words with different cases. For example, the words "Example" and "example" will be considered the same after lowercasing.
- **Stemming and lemmatization** Stemming is a technique that involves reducing words to their base or root form, by removing prefixes, suffixes, and other inflectional forms. The goal of stemming is to reduce the different forms of a word to a common base form so that similar words can be treated as equivalent for the purposes of text analysis. A more advanced form of stemming is lemmatization which aims to group words together based on their lemma, which is the base or dictionary form of a word. This technique utilizes the surrounding context and grammatical information to determine the appropriate lemma. For instance, while stemming would reduce

variations of a word like "changing", "changes" and "changed" to "chang" lemmatization would reduce them to their base form, "change".

- **Stop-words removal** Stop-words refers to words that carry no or little significance in text analysis. These words are often the most frequently occurring words among all texts in the text corpus, such as "a", "and" and "the".

3.2 Feature Representation

After the text has been pre-processed, the individual word tokens must be converted into a numerical representation that can be used as input in the classification models. This representation can take different forms, and two commonly used text representations are *Bag of Words* and *TF-IDF*, which are presented below.

3.2.1 Bag of Words

A simple way to represent text data is to use the bag of words model. Consider a text dataset, also called a *text corpus*, consisting of n texts, $T = [t_1, t_2, \dots, t_n]$. The set of all unique tokens w_d that occur in the text corpus, is called the *vocabulary*, $V = [w_1, w_2, \dots, w_D]$ (Allahtari et al., 2017, p. 4).

The bag of words model transforms each text instance in the text corpus into a vector, $\mathbf{x}_i = [x_{i1}, \dots, x_{iD}]$, where x_{id} represents the number of occurrences of a token w_d in text t_i (Sarkar, 2019, pp. 179–181). In the case of n texts and a vocabulary of size D , the bag of words model will be represented as a $n \times D$ matrix, as shown in Table 1, where each row represents a text instance and each column represents a token in the vocabulary.

Table 1: Bag of words matrix representation

	w_1	w_2	$\dots$	w_D
$\mathbf{x}_1$	x_{11}	x_{12}	$\dots$	x_{1D}
$\mathbf{x}_2$	x_{21}	x_{22}	$\dots$	x_{2D}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
$\vdots$	$\vdots$	$\vdots$		$\vdots$
$\mathbf{x}_n$	x_{n1}	x_{n2}	$\dots$	x_{nD}

As an example, consider a small text corpus, T_{ex} , consisting of the following two sentences:

t_1 this is an example
 t_2 this is another example

The vocabulary for the text corpus is: $V = [\text{this, is, an, example, another}]$, and the bag of words representation is found in Table 2.

Table 2: Bag of words representation of T_{ex}

	this	is	an	example	another
$\mathbf{x}_1$	1	1	1	1	0
$\mathbf{x}_2$	1	1	0	1	1

3.2.2 TF-IDF

A limitation of the bag of words model is that it treats all tokens equally, regardless of their frequency or rarity in the corpus. This means that tokens with high frequency will dominate the representation. (Sarkar, Bali & Sharma, 2017, p. 181) The *Term Frequency-Inverse Document Frequency (TF-IDF)* method, which was first introduced by Jones (1972), addresses this limitation by weighing the tokens based on their importance in a text. It assigns a high weight to tokens that are frequent in a particular text but rare across the entire text corpus. *TF-IDF* is the product of two metrics, *term frequency* (TF) and *inverse-document frequency* (IDF). (Sarkar, Bali & Sharma, 2017, p. 232)

The first metric, term frequency, measures how frequently a token w occurs in a given text instance t . Mathematically, it can be defined as:

$$TF(w, t) = \frac{f_{w,t}}{\sum_m f_{m,t}}, \quad (1)$$

where $f_{w,t}$ denotes the number of occurrences of the token w in text instance t and $\sum_m f_{m,t}$ is the total number of tokens in text instance t .

The second metric, inverse-document frequency, determines the weight of the tokens across all text instances in the text corpus and can be defined as:

$$IDF(w) = \ln \left(\frac{n}{df(w)} \right), \quad (2)$$

where n is the total number of text instances and $df(w)$ is the number of text instances that contains the token w in the text corpus.

Finally, *TF-IDF* is the product (1) and (2):

$$TF-IDF(w, t) = TF(w, t) \cdot IDF(w)$$

In Table 3, the *TF-IDF*-representation of the text corpus T_{ex} is presented. The term-frequency score for the tokens that occur in both $\mathbf{x}_1$ and $\mathbf{x}_2$ is $1/4 = 0.25$, while the inverse-document frequency is $\ln(2/2) = 0$, so $TF-IDF = 0.25 \cdot 0 = 0$. The term-frequency score for the tokens "an" and "another", that only occur in one of the text instances, is $1/4 = 0.25$, while the inverse-document frequency is $\ln(2/1) \approx 0.69$, therefore $TF-IDF = 0.25 \cdot 0.69 \approx 0.17$. In contrast to the bag of words representation in Table 2, where tokens that occurred in both $\mathbf{x}_1$ and $\mathbf{x}_2$ dominated the representation, the *TF-IDF*-weighting favors tokens that occur only in one of the texts.

Table 3: TF-IDF representation of T_{ex}

	this	is	an	example	another
$\mathbf{x}_1$	0.00	0.00	0.17	0.00	0.00
$\mathbf{x}_2$	0.00	0.00	0.00	0.00	0.17

3.3 Classification Model

3.3.1 Logistic Regression

The logistic regression model is a parametric method that belongs to the family of generalized linear models (GLM). It can be applied when the outcome variable is categorical and is a commonly used classification model in machine learning. In the binary classification setting, $y \in \{0, 1\}$, the aim is to estimate the conditional class probability of the positive output class $p(y = 1|\mathbf{x})$. The logistic regression model is linear in the log-odds:

$$\log \frac{p(y = 1|\mathbf{x})}{p(y = 0|\mathbf{x})} = \mathbf{x}^T \boldsymbol{\beta}$$

Since the aim is to model the conditional class probability $p(y = 1|\mathbf{x})$, the above equation can be rewritten as:

$$g(\mathbf{x}) = p(y = 1|\mathbf{x}) = \frac{e^{\mathbf{x}^T \boldsymbol{\beta}}}{1 + e^{\mathbf{x}^T \boldsymbol{\beta}}}$$

The above equation is the *logistic function* $h(z) = \frac{e^z}{1+e^z}$, applied to the linear combination $z = \mathbf{x}^T \boldsymbol{\beta}$. The logistic function ensures that the outcome is restricted to the interval $(0, 1)$.

There are several approaches to generalize the binary logistic regression to a setting where there are more than two outcome classes, $y \in \{1, \dots, k\}$. In this thesis, the multinomial logistic regression model will be based on the *softmax function*. Lindholm et al. (2022, p. 51–52) describes that when there are more than two outcome classes, the conditional class probabilities can be modeled by:

$$\mathbf{g}(\mathbf{x}) = \begin{bmatrix} g_1(\mathbf{x}) = p(y = 1|\mathbf{x}) \\ \vdots \\ g_K(\mathbf{x}) = p(y = k|\mathbf{x}) \end{bmatrix}$$

where each element $g_k(\mathbf{x})$ corresponds to the conditional class probability of y belonging to class k given the input $\mathbf{x}$. Since $\mathbf{g}(\mathbf{x})$ models a probability vector, each element needs to be larger than zero, and the sum of the vector need to sum to one.

Imagine K instances of $z = \mathbf{x}^T \boldsymbol{\beta}$, each denoted as z_k , and each with a different set of parameters $\boldsymbol{\beta}_k$ such that $z_k = \mathbf{x}^T \boldsymbol{\beta}_k$. If all z_k are stacked into a vector $\mathbf{z} = [z_1, \dots, z_K]^T$, the softmax function can be applied to normalize the vector $\mathbf{z}$:

$$\text{softmax}(\mathbf{z}) = \frac{1}{\sum_{k=1}^K e^{z_k}} \begin{bmatrix} e^{z_1} \\ \vdots \\ e^{z_K} \end{bmatrix}$$

The output of the softmax function will be a vector with the same dimensions as $\mathbf{z}$. The normalization ensures that the output vector will always sum to one, and each element will be larger than zero. The individual class probabilities, $p(y = k|\mathbf{x})$, are given by:

$$g_k(\mathbf{x}) = \frac{e^{\mathbf{x}^T \boldsymbol{\beta}_k}}{\sum_{j=1}^K e^{\mathbf{x}^T \boldsymbol{\beta}_j}} \quad k = 1, \dots, K \quad (3)$$

3.3.1.1 Parameter Estimation

The model parameters are estimated through maximum likelihood. In contrast to statistics, where the goal is to maximize the log-likelihood $l(\boldsymbol{\beta})$, it is in machine learning more common to minimize the negative log-likelihood, $J(\boldsymbol{\beta}) = -l(\boldsymbol{\beta})$, also called the cost function. The goal is to estimate:

$$\hat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\text{argmin}} J(\boldsymbol{\beta})$$

The likelihood of (3) is given by:

$$L(\boldsymbol{\beta}|y, \mathbf{x}) = \prod_{i=1}^n \prod_{k=1}^K \frac{e^{\mathbf{x}_i^T \boldsymbol{\beta}_k}}{\sum_{j=1}^K e^{\mathbf{x}_i^T \boldsymbol{\beta}_j}}$$

Thus, the cost function $J(\boldsymbol{\beta})$ can be expressed as:

$$\begin{aligned} J(\boldsymbol{\beta}) &= -\log L(\boldsymbol{\beta}) \\ &= -\sum_{i=1}^n \left[\sum_{k=1}^K \mathbb{I}\{y_i = k\} \log \frac{e^{\mathbf{x}_i^T \boldsymbol{\beta}_k}}{\sum_{j=1}^K e^{\mathbf{x}_i^T \boldsymbol{\beta}_j}} \right] \\ &= \sum_{i=1}^n \sum_{k=1}^K \mathbb{I}\{y_i = k\} \left[-\mathbf{x}_i^T \boldsymbol{\beta}_k + \log \left(\sum_{j=1}^K e^{\mathbf{x}_i^T \boldsymbol{\beta}_j} \right) \right] \end{aligned} \quad (4)$$

The inclusion of the indicator function,

$$\mathbb{I} = \begin{cases} 1, & \text{if } y_i = k \\ 0, & \text{otherwise} \end{cases}$$

accounts for the fact that each input vector $\mathbf{x}_i$ can only belong to one of the K classes.

The multinomial logistic regression model (3) is overparameterized, meaning that there is no unique set of parameters that minimizes the cost function. Therefore, it is common to add a regularization term to the cost function, $J(\boldsymbol{\beta})$. This ensures, despite the overparameterization, that there exists a unique solution to the optimization problem. Regularization also prevents overfitting by keeping the parameter values small, unless the data strongly supports large parameter values. This is particularly important in the case of text classification since the nature of text data introduces a high number of features, which increases the risk of overfitting. There exist different types of regularization methods, but for the logistic regression model, it is most common to use the $L2$ -penalty. (Lindholm et al., 2022, pp. 56–57) By adding the penalty to the cost function in (4), it can be expressed as:

$$J(\boldsymbol{\beta}) = \sum_{i=1}^n \sum_{k=1}^K \mathbb{I}\{y_i = k\} \left[-\mathbf{x}_i^T \boldsymbol{\beta}_k + \log\left(\sum_{j=1}^K e^{\mathbf{x}_i^T \boldsymbol{\beta}_j}\right) \right] + \lambda \|\boldsymbol{\beta}\|_2^2,$$

where $\lambda \geq 0$ is the regularization parameter that controls the strength of the penalty and $\|\boldsymbol{\beta}\|_2^2 = \sum_i^n \boldsymbol{\beta}_i^2$.

Since there is no closed-form solution for this maximization problem, a numerical optimization method must be applied to estimate the model parameters. A commonly used method is *Gradient Descent*. (Lindholm et al., 2022, p. 116) The gradient of the cost function:

$$\nabla_{\boldsymbol{\beta}} J(\boldsymbol{\beta}) = \frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}_j},$$

describes the direction of the steepest ascent, i.e. the direction in which $J(\boldsymbol{\beta})$ increases. Likewise, $-\nabla_{\boldsymbol{\beta}} J(\boldsymbol{\beta})$ describes the direction in which $J(\boldsymbol{\beta})$ decreases. By taking a small step in the direction of the negative gradient, the value of $J(\boldsymbol{\beta})$ will be reduced:

$$J(\boldsymbol{\beta} - \alpha \nabla_{\boldsymbol{\beta}} J(\boldsymbol{\beta})) \leq J(\boldsymbol{\beta})$$

where the parameter $\alpha > 0$ is the step size. For convex objective functions, this inequality is strict except at the minimum where $\nabla_{\boldsymbol{\beta}} J(\boldsymbol{\beta}) = 0$. To select $\boldsymbol{\beta}^{(t+1)}$ such that $J(\boldsymbol{\beta}^{(t+1)}) \leq J(\boldsymbol{\beta}^{(t)})$, the update rule for $\boldsymbol{\beta}^{(t)}$ can be specified as:

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \alpha \sum_{i=1}^n \nabla_{\boldsymbol{\beta}} J_i(\boldsymbol{\beta}) \quad (5)$$

The gradient descent algorithm is obtained by repeating (5).

Since large datasets, such as text data, require more computational power, the computation of the gradient using the gradient descent algorithm can be slow. Therefore it is common to use *Stochastic Gradient Descent (SGD)*, which instead

of computing the gradient using the entire dataset, only computes it based on a fraction of it. There are many different extensions of stochastic gradient descent, and for logistic regression, the *Stochastic Average Gradient (SAG)* has demonstrated good results. (Lindholm et al., 2022, pp. 124–126)

The stochastic average gradient algorithm was introduced by Roux, Schmidt & Bach (2012). The idea behind this algorithm is to maintain an estimate of $\mathbf{v}_i^{(t)} = \nabla_{\boldsymbol{\beta}} J_i(\boldsymbol{\beta})$ for every data point $i = 1, \dots, n$. The average of $\mathbf{v}_i$ is used to update the parameters of the model. The update rule according to the stochastic average gradient can be specified as:

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \frac{\alpha_t}{n} \sum_{i=1}^n \mathbf{v}_i^{(t)},$$

where α_t is the step size. An index i_t is randomly selected at each iteration to choose which sample to compute the gradient for. All other $\mathbf{v}_i^{(t)}$ are kept at their previous value. That is:

$$\mathbf{v}_i^{(t)} = \begin{cases} \nabla_{\boldsymbol{\beta}} J_i(\boldsymbol{\beta}^{(t)}) & \text{if } i = i_t \\ \mathbf{v}_i^{(t-1)} & \text{otherwise} \end{cases} \quad (6)$$

By storing the gradients for every data point, these can be reused in future iterations, which can lead to faster convergence and improved computational efficiency compared to other stochastic gradient descent algorithms (Roux, Schmidt & Bach, 2012).

3.3.2 Multinomial Naive Bayes

The *Naive Bayes* algorithm is a commonly used classification model in text classification. The "naive" part of the model is that it assumes that the presence of a particular feature in a class is independent of the presence of any other features. While this assumption is false in most real-world scenarios, the naive Bayes classifier still performs remarkably well in classification tasks. (McCallum & Nigam, 1998) The model is based on Bayes' theorem which states that given a response class label k and a set of features $\mathbf{x}_i$, the conditional class probability of y_i given the features can be expressed as:

$$P(y_i = k | \mathbf{x}_i) = \frac{P(\mathbf{x}_i | y_i = k) \cdot P(y_i = k)}{P(\mathbf{x}_i)}, \quad (7)$$

where $P(\mathbf{x}_i | y_i = k)$ is the probability of the features $\mathbf{x}_i$ given the label, $P(y_i = k)$ is the prior probability of y_i belonging to class k and $P(\mathbf{x}_i)$ is the probability of observing the features.

Since the denominator in (7) only scales the posterior probabilities by the same quantity for all classes, it can be removed from the expression without affecting the relative ordering of the predicted class probabilities. Thus, the expression can be simplified as follows:

$$P(y_i = k | \mathbf{x}_i) \propto P(\mathbf{x}_i | y_i = k) \cdot P(y_i = k) \quad (8)$$

In a bag of words model, as described in Section 3.2.1, each text instance t_i is represented by a feature vector $\mathbf{x}_i$, where $x_{i,d}$ represents the number of occurrences of tokens w_d in text t_i . When tokens are represented as the number of occurrences (rather than binary values), the *Multinomial Naive Bayes* algorithm is applied, which assumes that the occurrence of each token in a text instance is independent of the occurrences of other tokens. As a result, each text instance t_i can be modeled as being generated from a multinomial distribution of tokens. (McCallum & Nigam, 1998) The first part of (8) can then be expressed as:

$$P(\mathbf{x}_i | y_i = k) = \frac{(\sum_d^D x_{i,d})!}{\prod_d^D x_{i,d}!} \prod_d^D P(w_d | y_i = k)^{x_{i,d}}, \quad (9)$$

where the first term on the right-hand side is the multinomial coefficient. The second term on the right-hand side, $P(w_d | y)$, is the probability of token w_d given label k which can be estimated by:

$$\hat{P}(w_d | y = k) = \frac{\sum_i^n x_{i,d} \cdot \mathbb{I}\{y_i = k\}}{\sum_{d=1}^D \sum_{i=1}^n x_{i,d} \cdot \mathbb{I}\{y_i = k\}}, \quad (10)$$

where the numerator is the sum of occurrences of token w_d from all texts in the training set that belongs to class k . The denominator is the sum of all token occurrences in the training set that belongs to class k .

A problem with (10) is that it assigns zero probability to tokens that do not occur in text instance t_i . To avoid zero probabilities, a smoothing term η can be applied. The two most common variants of smoothing are the *Laplace smoothing* ($\eta = 1$) and *Lidstone smoothing* ($\eta < 1$). (Raschka, 2014) Adding the smoothing term to (10) yields:

$$\hat{P}(w_d | y_i = k) = \frac{\sum_i^n x_{i,d} \cdot \mathbb{I}\{y_i = k\} + \eta}{\sum_{d=1}^D \sum_i^n x_{i,d} \cdot \mathbb{I}\{y_i = k\} + \eta \cdot D}.$$

The second part of (8), i.e. the prior probability, can be estimated as the proportion of text instances labeled with k in the training set (Raschka, 2014):

$$\hat{P}(y_i = k) = \frac{n_k}{n},$$

where n_k is the number of instances belonging to class k and n is the total number of observations.

4 Data

Active learning can be applied in various text classification tasks, particularly when there is a large amount of unlabeled text data available and the aim is to reduce the labeling effort. Some examples of possible applications are customer sentiment classification, web page classification, and document classification.

Since the purpose of this thesis is to demonstrate the effect of applying uncertainty sampling, the application will be on already labeled data. In the following sections, the two datasets used in this thesis will be presented. While these datasets may not be completely representative of real-life text classification tasks, they demonstrate the potential of applying active learning to different types of texts that vary in both text content and text length.

4.1 Students Questions Dataset

The first dataset used in this thesis is the "IITJEE NEET AIIMS Students Questions Dataset", found on Kaggle.com (Biswal, 2020). The dataset consists of 122 519 questions, both open and multiple-choice, from past entrance exams, such as IIT-JEE, NEET, and AIIMS, which are all highly competitive exams that students take to get admission into one out of India's most prestigious universities.

The questions are classified into one out of four different subjects: "Biology", "Chemistry", "Mathematics", or "Physics". The number of questions labeled with each subject is shown in Figure 5 below.

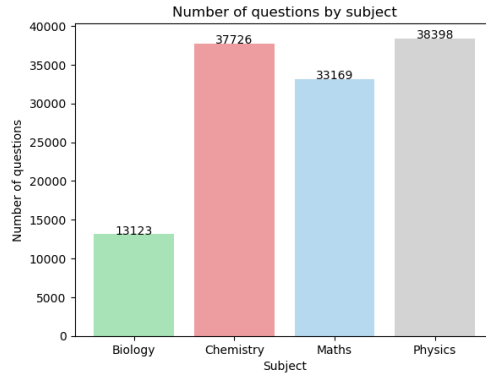


Figure 5: Number of questions by subject

In Figure 6, the distribution of the token counts per question is presented. Most questions are shorter than 100 tokens, and the average number of tokens in each question is 69.

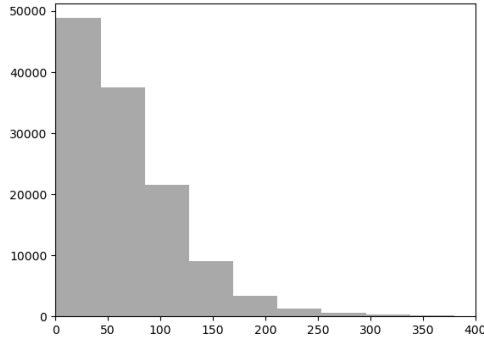


Figure 6: Distribution of token counts per question

4.2 E-commerce Dataset

The second dataset is the "E-commerce Text Classification Dataset" (Gautam, 2019). The dataset consists of 50 424 product descriptions scraped from an Indian e-commerce platform. Each product description is categorized into one out of four categories: "Books", "Clothing & Accessories", "Electronics", and "Household". The number of descriptions labeled with each category is shown in Figure 7.

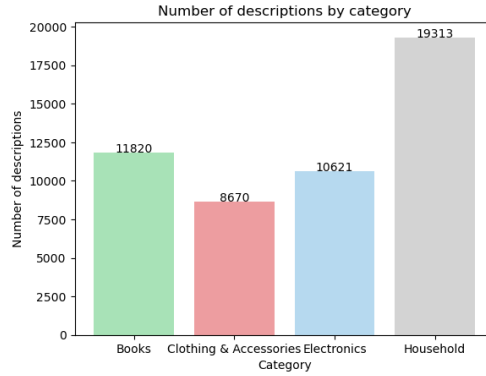


Figure 7: Number of descriptions by category

In Figure 8, the distribution of the token counts in the product descriptions is presented. Most descriptions are shorter than 180 tokens, and the average number of tokens in each description is 132.

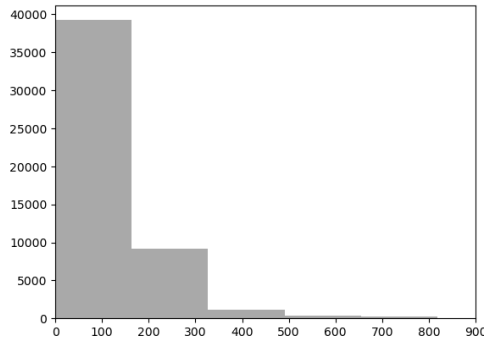


Figure 8: Distribution of token counts per product description

5 Method and Implementation

In this chapter, the details of the method and the implementation of the active learning cycle are presented. The implementation is done using the Python programming language.

It is important to note that in this study, the labeling process of the active learning cycle is mimicked since the datasets used are already fully labeled. This approach makes it possible to explore the impact of uncertainty sampling on the performance of the classification models, specifically in comparison to random sampling, without the need for an actual human annotator.

5.1 Split

The datasets are initially split into a *training* and *test* set. For the test set, which is used to evaluate the performance of the query strategies (see Section 5.5), 20% of the total number of observations is randomly drawn without replacement from each dataset. The rest of the observations, denoted as the training set $\mathcal{D}$, are used in the active learning cycle presented in Section 5.6.

5.2 Pre-Processing

Both datasets are pre-processed to remove characters and tokens that do not provide any meaningful information for the classification models. The same pre-processing steps are applied to both the training and test set.

For both datasets, tokenization is performed such that each token is any sequence of two or more alphanumeric characters that are surrounded by word boundaries, i.e. the position where an alphanumeric character is not followed or preceded by another alphanumeric character. This is done using `regexp` in *Natural Language Toolkit* (NLTK) (NLTK, 2023b).

Since the questions in the students questions dataset are written in LaTeX format, the data contains a lot of special characters. Therefore, the data is cleaned by removing all non-alphanumeric characters. Also, all digits in the questions are transformed into the text string "digit". This is done to reduce the vocabulary size and at the same time retain information regarding if any digit was present in the questions. Since the majority of the questions are multiple-choice, characters indicating which answer to choose (such as "A", "B" or "C") occur with high frequency and are therefore removed from the data. In addition, since some tokens occurred with high frequency in all subjects, the ten most common tokens, excluding the token "digit", are removed. A list of these can be found in Appendix A.1. The e-commerce dataset is cleaned by removing non-alphanumeric characters and digits. After cleaning the datasets, all letters are converted to lowercase.

Stop-words removal is performed using `corpus` in *NLTK* (NLTK, 2023a), which provides a list of common English stop-words, presented in Appendix A.2. Lastly, the tokens are lemmatized using `wordnet` in the *NLTK* library. (NLTK, 2023c)

For the students questions dataset, there were 74 texts from the training set and 29 texts from the test set that only consisted of non-alphanumeric characters, and

after applying the pre-processing these texts were blank and were removed from the dataset. Therefore, the final dataset consists of 122 416 texts.

Each pre-processing step results in a reduction of the vocabulary size. In Table 4 the vocabulary size after applying each pre-processing step for both datasets is presented.

Table 4: Vocabulary size after applying each pre-processing step

Pre-processing step	Students Questions	E-commerce
Noise removal	43259	84729
Lower casing	33289	61527
Stop-words removal	33147	61385
Lemmatization	29960	55785

5.3 Feature Representation

Following the pre-processing, the data needs to be converted to a numerical representation. For the multinomial logistic regression model, the text instances are transformed into a TF-IDF representation using `TfidfVectorizer` in the *scikit-learn* library (Scikit-learn, 2023b). This function calculates a slightly modified version of the one explained in Section 3.2.2, since it adds 1 in the calculation of $IDF(w)$. That is:

$$IDF(w) = \ln \left(\frac{n}{df(w)} \right) + 1 \quad (11)$$

This means that if there are tokens that occur in all texts, these will be assigned a small weight instead of being completely ignored.

Since the multinomial naive Bayes model is based on the assumption of tokens being represented as occurrences, the text instances are also transformed into a bag of words representation, as explained in Section 3.2.1, using the `CountVectorizer` function from the *scikit-learn* library (Scikit-learn, 2023a).

5.4 Classification Models

The function `LogisticRegression`, available in the *scikit-learn* library is used to implement the multinomial logistic regression model (Scikit-learn, 2023c). The parameter estimation is done using the $L2$ -penalty, where the regularization parameter is set to the default setting $\lambda = 1$. The solver is set to "SAG", meaning that the parameters are estimated using the stochastic average gradient algorithm.

The multinomial naive Bayes model is implemented using the `MultinomialNB` function in the *scikit-learn* library (Scikit-learn, 2023d). The smoothing parameter is set to $\eta = 1$, meaning that Laplace smoothing is applied when estimating the probabilities.

5.5 Performance Metrics

The classification models are used to predict the label $\tilde{y}$ for the test data, denoted as $\tilde{\mathbf{x}}$. The prediction of $\tilde{y}$ will be to the most probable class according to the model, that is:

$$\tilde{y} = \underset{k}{\operatorname{argmax}} P(y = k | \tilde{\mathbf{x}}).$$

To evaluate the performance of the classification models, the counts of the correctly and incorrectly classified instances predicted by the model can be used. These can be summarized by a 4×4 confusion matrix as presented in Figure 9. The columns in the confusion matrix represent the predicted classes by the model, while the rows represent the actual classes. The diagonal elements, TP_k , are the number of correctly classified instances in the k th class, while the off-diagonal elements are the number of misclassified instances. For example, $E_{1,2}$ represents the number of instances misclassified as 2 when the true class is 1.

		Predicted			
		$\tilde{y} = 1$	$\tilde{y} = 2$	$\tilde{y} = 3$	$\tilde{y} = 4$
Actual	$y = 1$	TP_1	$E_{1,2}$	$E_{1,3}$	$E_{1,4}$
	$y = 2$	$E_{2,1}$	TP_2	$E_{2,3}$	$E_{2,4}$
	$y = 3$	$E_{3,1}$	$E_{3,2}$	TP_3	$E_{3,4}$
	$y = 4$	$E_{4,1}$	$E_{4,2}$	$E_{4,3}$	TP_4

Figure 9: 4×4 confusion matrix

There are several different performance metrics that can be derived from the confusion matrix. In this thesis, accuracy will be used to track the model's performance after every newly added instance to $\mathcal{L}$ during the active learning cycle. This is often referred to as a *learning curve*, which is a common way to evaluate active learning strategies (Settles, 2010, p. 7). Accuracy is the percentage of correctly predicted instances by the model and can be defined as:

$$A = \frac{\sum_{i=k}^4 TP_k}{\tilde{n}},$$

where $\tilde{n}$ is the total number of observations in the test data.

Since the datasets are slightly imbalanced, meaning that the distribution of the classes is uneven, accuracy alone may not provide a complete picture of the models' performance. Therefore, the *F1-score* will be calculated in addition to accuracy,

which is the harmonic mean of two metrics, *precision* and *recall*. Precision measures how many of the instances predicted to belong to a particular class actually belong to that class. In other words, it measures the fraction of true positives (TP_k) out of all the instances classified by the model to belong to that class:

$$\text{Precision}_k = \frac{TP_k}{\sum_{i=1}^{\tilde{n}} \mathbb{I}\{\tilde{y}_i = k\}}, \quad (12)$$

where $\sum_{i=1}^{\tilde{n}} \mathbb{I}\{\tilde{y}_i = k\}$ represents the number of instances that the model predicts to belong to class k .

Recall, on the other hand, measures how many of the text instances that actually belong to a particular class are correctly identified by the model. In other words, it measures the fraction of true positives (TP_k) out of all the instances that actually belong to that class:

$$\text{Recall}_k = \frac{TP_k}{\sum_{i=1}^{\tilde{n}} \mathbb{I}\{y_i = k\}}, \quad (13)$$

where $\sum_{i=1}^{\tilde{n}} \mathbb{I}\{y_i = k\}$ denotes the actual number of instances belonging to class k .

The F1-score for class k , which is the harmonic mean between (12) and (13) is then defined as:

$$\text{F1}_k = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}.$$

To obtain an overall metric for F1, both micro-averaging and macro-averaging can be used. Micro-averaging favors larger classes, while macro-averaging treats all classes with equal importance. (Sokolova & Lapalme, 2009) In this thesis, the performance of all classes is equally important, and therefore macro-averaging is used, which is defined as the average of the F1-score obtained for all classes:

$$\text{F1}_{\text{macro}} = \frac{1}{4} \sum_{k=1}^4 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}.$$

Due to computational limitations, the F1-score is calculated only at 200, 1000 and 2000 labeled instances, and not for every added instance as for accuracy. However, measuring the F1-score at these measure points will still give a good overview of the performance of the query strategies.

5.6 The Active Learning Cycle

To initialize the labeled dataset $\mathcal{L}_0$, 100 observations are randomly drawn without replacement from the training set $\mathcal{D}$. The unlabeled pool consists of the remaining data points, $\mathcal{U}_0 = \mathcal{D} - \mathcal{L}_0$. The initial setup for each dataset can be found in Table 5.

Table 5: Initial setup for both datasets

Dataset	Test	$\mathcal{U}_0$	$\mathcal{L}_0$
Students questions	24504	97015	100
E-commerce	10085	40239	100

Since both datasets used in this thesis are fully labeled, the labeling process can be simulated without the need for a human annotator. Firstly, random sampling is implemented as described in Algorithm 1. The algorithm starts by training an initial model $\mathcal{M}$ on $\mathcal{L}_0$. The model is then applied on $\mathcal{U}_0$ to calculate the initial accuracy of the model, A_0 . The algorithm adds new observations to $\mathcal{L}$ by sampling them at random without replacement from the unlabeled pool. The sampled instance is then added to $\mathcal{L}$ and removed from $\mathcal{U}$. Then, the model is retrained on $\mathcal{L}$ to incorporate the information from the newly added instance. Finally, the iteration ends by calculating and storing the accuracy A_t . The labeling budget is set to 2000, meaning that the labeling process is repeated 1900 times, until there are 2000 labeled instances in $\mathcal{L}$. This was determined considering both the computational time and the need to have a reasonable number of labeled instances to examine the impact of the query strategies.

Algorithm 1 Random Sampling

```

 $\mathcal{U}_0 = \{\mathbf{x}_u, y_u\}_{u=1}^U$ 
 $\mathcal{L}_0 = \{\mathbf{x}_l, y_l\}_{l=1}^L$ 
 $\mathcal{M} = \text{train}(\mathcal{L}_0)$ 
Calculate the initial accuracy  $A_0$ 
for  $t = 1, 2, \dots, 1900$  do
    Draw  $\mathbf{x}_i$  from  $\mathcal{U}_t$  without replacement
    Add  $\{\mathbf{x}_i, y_i\}$  to  $\mathcal{L}$ 
    Remove  $\{\mathbf{x}_i, y_i\}$  from  $\mathcal{U}$ 
     $\mathcal{M} = \text{train}(\mathcal{L})$ 
    Calculate the accuracy  $A_t$ 
end for

```

The uncertainty sampling query strategies are implemented as described in Algorithm 2. The algorithm starts, just as for random sampling by training an initial model on $\mathcal{L}_0$. The model $\mathcal{M}$ is then applied on $\mathcal{U}_0$ to calculate the initial accuracy of the model, A_0 . In each iteration, the estimated conditional class probabilities are used to decide which instance is most uncertain, according to the query strategies described in Section 2.2.2. The most uncertain instance, according to each strategy, is then selected and added to $\mathcal{L}$, and removed from $\mathcal{U}$. Then, the model is retrained and the accuracy is re-calculated and recorded. Just as for the random sampling strategy, the process is repeated 1900 times.

Algorithm 2 Uncertainty Sampling

```
 $\mathcal{U}_0 = \{\mathbf{x}_u, y_u\}_{u=1}^U$   
 $\mathcal{L}_0 = \{\mathbf{x}_l, y_l\}_{l=1}^L$   
 $\mathcal{M} = \text{train}(\mathcal{L}_0)$   
Calculate the initial accuracy  $A_0$   
for  $t = 1, 2, \dots, 1900$  do  
    Calculate the predicted class probabilities for each unit in  $\mathcal{U}_t$   
    Select the most uncertain text instance  $\mathbf{x}_i^*$  from  $\mathcal{U}_t$  according to the query  
    strategy  
    Add  $\{\mathbf{x}_i^*, y_i^*\}$  to  $\mathcal{L}$   
    Remove  $\{\mathbf{x}_i^*, y_i^*\}$  from  $\mathcal{U}$   
     $\mathcal{M} = \text{train}(\mathcal{L})$   
    Calculate the accuracy  $A_t$   
end for
```

To account for the fact that the performance of the query strategies is dependent on the initial labeled set, the algorithms are repeated ten times for each query strategy, each time with a different initial set of labeled instances $\mathcal{L}_0$. The final results, presented in the next section, are the average accuracy and F1-scores obtained over the ten runs. Ideally, a larger number of repetitions would have been preferred to obtain more reliable results. However, due to the significant computational power required, the study is limited to ten runs.

6 Result

In this section, the results from the experiments will be presented. Firstly, the results based on the students questions dataset will be presented, followed by the results based on the e-commerce dataset. The initial models are trained on 100 labeled instances. A horizontal line representing the accuracy and F1-score obtained by training the models on the full training set $\mathcal{D}$ will be included in each plot as a reference.

6.1 Students Questions Dataset

Figure 10 presents the average test set accuracy for each query strategy. To the left, the result from the logistic regression model is presented, and to the right, the result from the multinomial naive Bayes model is presented.

For the logistic regression model, the initial accuracy is around 50%. The uncertainty sampling strategies outperform random sampling from the start. Margin sampling performs the best and achieves around the same accuracy after 600 labeled instances as random sampling does after 2000 labeled instances. Least confident sampling and entropy sampling perform approximately the same during the whole active learning cycle. For the multinomial naive Bayes model, the initial accuracy is around 60%. Margin sampling performs the best and achieves around the same accuracy after 700 labeled instances as random sampling does after 2000 labeled instances. Least confident sampling and entropy sampling initially perform worse than random sampling, but after 1100 labeled instances they surpass random sampling. Least confident sampling reaches a slightly higher accuracy compared to entropy sampling after 2000 labeled instances.

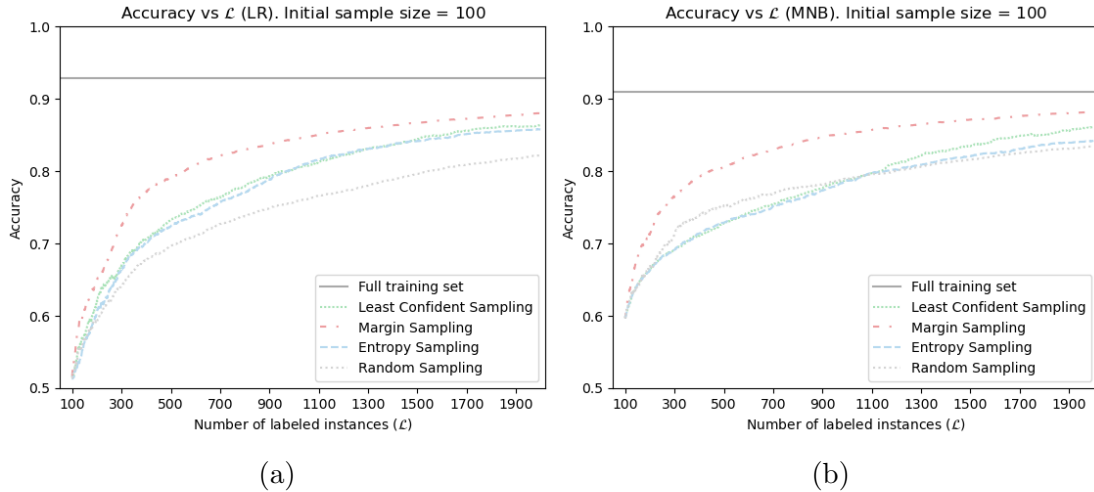


Figure 10: (a) Average test set accuracy vs the number of labeled instances for the logistic regression model. (b) Average test set accuracy vs the number of labeled instances for the multinomial naive Bayes model.

In addition to accuracy, the average test set F1-score was calculated, which is presented in Figure 11. For the logistic regression model (a), least confident sampling results in the highest score at 200 labeled instances. However, at 1000 and

2000 labeled instances, margin sampling results in the highest score. Random sampling results in the lowest score at 200, 1000 and 2000 labeled instances. For the multinomial naive Bayes model (b), both least confident sampling and entropy sampling results in the highest score at 200 labeled instances, while random sampling results in the lowest. At 1000 and 2000 labeled instances, however, margin sampling results in the highest score.

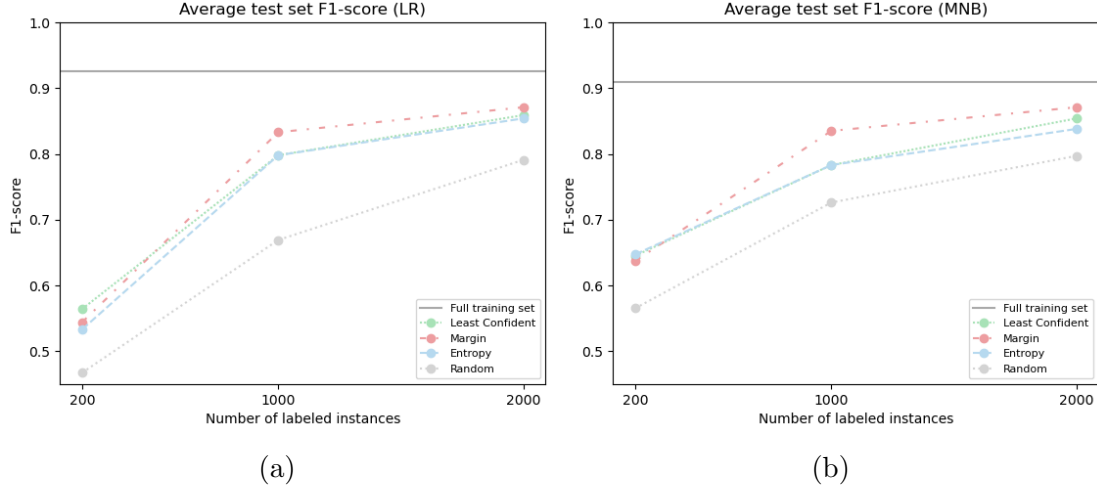


Figure 11: (a) Average test set F1-score at 200, 1000 and 2000 labeled instances for the logistic regression model. (b) Average test set F1-score at 200, 1000 and 2000 labeled instances for the multinomial naive Bayes model.

6.2 E-commerce Dataset

In Figure 12 below, the average test set accuracy obtained for each query strategy is presented. To the left, the result from the logistic regression model is presented, and to the right, the result from the multinomial naive Bayes model is presented.

For the logistic regression model, the initial accuracy is around 50%. All uncertainty sampling strategies perform approximately the same for the first 100 labeled instances. For the rest of the queried instances, margin sampling achieves the highest accuracy, followed by least confident sampling. After 2000 labeled instances, the uncertainty sampling strategies result in almost the same accuracy as when training the model on the full training set. Also, they result in about the same accuracy after 300 labeled instances as random sampling does after 2000 labeled instances. For the multinomial naive Bayes model, the initial accuracy is around 75%. Margin sampling converges the fastest and is after 500 labeled instances reaching the same accuracy that random sampling does after 2000 labeled instances. Least confident sampling performs about the same as random sampling until 500 labeled instances and after 1100 labeled instances, the performance is about the same as for the margin sampling strategy. Entropy sampling performs worse than random sampling until 700 labeled instances and is after 1700 labeled instances reaching the same level of accuracy as the other two uncertainty sampling strategies. After 2000 labeled instances, all uncertainty sampling strategies yield a slightly higher accuracy compared to training the model on the full training

set.

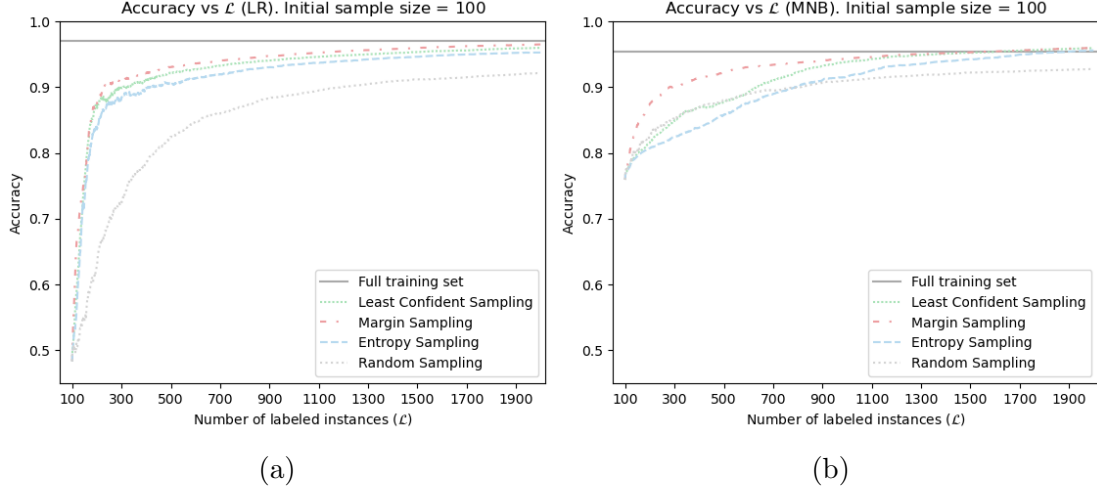


Figure 12: (a) Average test set accuracy vs the number of labeled instances for the logistic regression model. (b) Average test set accuracy vs the number of labeled instances for the multinomial naive Bayes model.

In Figure 13, the average test set F1-score is presented. For the multinomial logistic regression model (a), margin sampling results in the highest score, closely followed by least confident sampling. All uncertainty strategies result in a higher F1-score compared to random sampling, especially at 200 labeled instances. For the multinomial naive Bayes model (b), only margin sampling results in a higher F1-score compared to random sampling at 200 labeled instances. At 1000 and 2000 labeled instances, however, all uncertainty sampling strategies results in a higher score compared to random sampling.

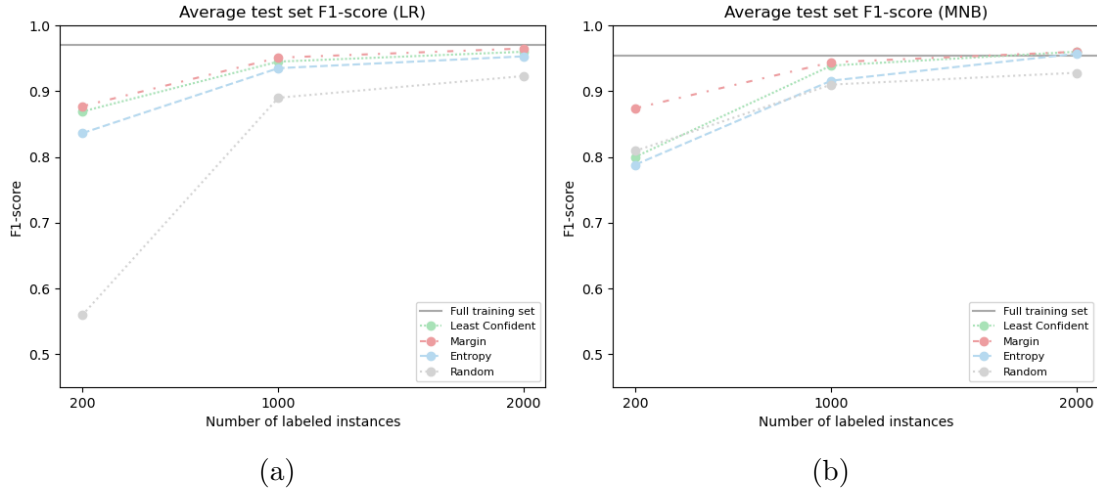


Figure 13: (a) Average test set F1-score after 200, 1000 and 2000 labaled instances for the logistic regression model. (b) Average test set F1-score after 200, 1000 and 2000 labeled instances for the multinomial naive Bayes model.

7 Discussion

The findings in the previous section indicate that applying active learning, and more specifically uncertainty sampling, can produce a model with better performance compared to random sampling under a fixed labeling budget. This implies that uncertainty sampling can reduce the labeling effort needed to obtain a model with good performance. However, it was shown that the performance is dependent on the classification model and the query strategy used.

Margin sampling consistently outperformed the other uncertainty sampling query strategies in all experiments when it comes to both accuracy and the F1-score. That margin sampling performed better compared to least confident sampling is expected since it considers both the most and second most probable label. However, that entropy sampling, which considers the entire distribution of predicted conditional class probabilities, performed the worst was unexpected. A possible explanation of this result could be that entropy sampling favors instances where the predicted probabilities are evenly spread, which results in potentially selecting instances where the classification model is uncertain about several classes. In contrast, margin sampling favors instances where the model is especially uncertain about two classes, which for this particular application seems to capture uncertainty the best. Applying uncertainty sampling to a dataset with either fewer or more outcome classes, however, could yield a different result from the one obtained in this thesis.

The findings also suggest that the performance of uncertainty sampling strategies differs depending on which classification model is used. When combining active learning with logistic regression, all uncertainty sampling strategies outperformed random sampling already from the first queried instances. However, when combining active learning with multinomial naive Bayes, only margin sampling resulted in a significantly higher accuracy during the whole learning process. Least confident sampling and entropy sampling, on the other hand, initially performed worse than random sampling, but then eventually surpassed it after a relatively large number of labeled instances.

Another finding is that when using uncertainty sampling using only a fraction of the dataset, the performance of the classification model was similar, and sometimes even higher, compared to when training the model using the full training set. This indicates that active learning can not only minimize the labeling effort but also produce more effective training sets compared to using the full training set.

It is important to note that in a real-life application, there are aspects of active learning that have not been considered in this thesis. For example, with an actual human annotator, there will be costs associated with labeling each instance and in this scenario, it would not be realistic to label only one instance at a time. Therefore, it would be interesting for future research to investigate the impact of querying instances in batches of b instances rather than querying only one instance at a time. This would speed up the computational time since it would allow more efficient labeling. However, it is not clear how this approach would affect the performance of the query strategies. Therefore, it would be interesting to investigate the trade-off between computational efficiency and the performance

of the query strategies in this context. It would also be interesting to apply the uncertainty sampling strategies tested in this thesis to a classification problem with more than four possible labels and investigate how this affects the performance of the query strategies.

8 Conclusions

In this thesis, the main objective was to investigate the impact of applying uncertainty sampling, in a text classification setting. In addition, the impact of various uncertainty sampling strategies and classification models was explored.

It was found that margin sampling consistently outperformed both random sampling and the other uncertainty sampling strategies. Another finding was that the effectiveness of uncertainty sampling strategies may vary depending on the type of classification model used, particularly for least confident sampling and entropy sampling. Finally, the study found that a model trained on only a fraction of the training data obtained from uncertainty sampling could produce a model with better performance compared to training the model using the full training set.

The reported findings were based on applying active learning to only two different datasets, which limits the generalizability. Further research is needed to confirm the effectiveness of uncertainty sampling strategies across a wider range of datasets and classification models.

Bibliography

- Allahyari, M., Pouriyeh, S., Assefi, M., Safaei, S., Trippe, E., Gutiérrez, J. & Kochut, K. (2017), ‘A brief survey of text mining: Classification, clustering and extraction techniques’. Available from: doi:10.48550/arXiv.1707.02919.
- Biswal, M. (2020), ‘Iitjee-neet-aims students questions data’, Available from: <https://www.kaggle.com/datasets/mrutyunjaybiswal/iitjee-neet-aims-students-questions-data>. [Accessed: 6th March, 2023].
- Fedorov, V. (1972), *Theory of Optimal Experiments Designs*, New York, Academic Press.
- Gautam (2019), ‘E commerce text dataset (version - 2)’, Available from: doi: 10.5281/zenodo.3355823. [Accessed: 15th March, 2023].
- Jones, K. S. (1972), ‘A statistical interpretation of term specificity and its application in retrieval’, *Journal of Documentation* **28**(1), 11–21.
- Joshi, A. J., Porikli, F. & Papanikolopoulos, N. (2009), Multi-class active learning for image classification, in ‘2009 IEEE Conference on Computer Vision and Pattern Recognition’, pp. 2372–2379.
- Kiefer, J. (1959), ‘Optimum experimental designs’, *Journal of the Royal Statistical Society. Series B (Methodological)* **21**(2), 272–319.
- Körner, C. & Wrobel, S. (2006), Multi-class ensemble-based active learning, in ‘Machine Learning: ECML 2006: 17th European Conference on Machine Learning Berlin, Germany, September 18–22, 2006 Proceedings 17’, Springer, pp. 687–694.
- Lang, K. & Baum, E. (1992), ‘Query learning can work poorly when a human oracle is used’, *Proceedings of the IEEE International Joint Conference on Neural Networks* pp. 335–340.
- Lewis, D. D. & Gale, W. A. (1994), A sequential algorithm for training text classifiers, in B. W. Croft & C. J. van Rijsbergen, eds, ‘SIGIR ’94’, Springer London, London, pp. 3–12.
- Lindholm, A., Wahlström, N., Lindsten, F. & Schön, T. B. (2022), *Machine Learning - A First Course for Engineers and Scientists*, Cambridge University Press. Available from: <https://smlbook.org> [Accessed 15th February 2023].
- McCallum, A. & Nigam, K. (1998), A comparison of event models for naive bayes text classification, in ‘AAAI Conference on Artificial Intelligence’.
- Mitchell, T. (1997), *Machine Learning*, McGraw-Hill International Editions, McGraw-Hill.
- NLTK (2023a), ‘NLTK Corpus’, Available from: <https://www.nltk.org/api/nltk.corpus>. [Accessed 24th March 2023].
- NLTK (2023b), ‘NLTK Tokenize’, Available from: <https://www.nltk.org/api/nltk.tokenize.regexp.html>. [Accessed 24th March 2023].

- NLTK (2023c), ‘NLTK WordNet’, Available from: https://www.nltk.org/_modules/nltk/stem/wordnet.html. [Accessed 24th March 2023].
- Raschka, S. (2014), ‘Naive Bayes and Text Classification I - Introduction and Theory’, Available from: doi:10.48550/arXiv.1410.5329 .
- Roux, N., Schmidt, M. & Bach, F. (2012), A stochastic gradient method with an exponential convergence rate for finite training sets, *in* ‘Advances in Neural Information Processing Systems’, Vol. 25, Curran Associates, Inc.
- Sarkar, D. (2019), *Text Analytics with Python: A Practitioner’s Guide to Natural Language Processing*, Burlington, Apress.
- Sarkar, D., Bali, R. & Sharma, T. (2017), *Practical Machine Learning with Python: A Problem-Solver’s Guide to Building Real-World Intelligent Systems*, Burlington, Apress.
- Scheffer, T., Decomaine, C. & Wrobel, S. (2001), Active hidden markov models for information extraction, *in* ‘Proceedings of the International Conference on Advances in Intelligent Data Analysis (CAIDA)’, Springer-Verlag, pp. 309–318.
- Scikit-learn (2023a), ‘sklearn.feature_extraction.text.countvectorizer’, Available from: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html. [Accessed 24th March 2023].
- Scikit-learn (2023b), ‘sklearn.feature_extraction.text.tfidfvectorizer’, Available from: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html. [Accessed 24th March 2023].
- Scikit-learn (2023c), ‘sklearn.linear_model.logisticregression’, Available from: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression. [Accessed 24th March 2023].
- Scikit-learn (2023d), ‘sklearn.naive_bayes’, Available from: https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html. [Accessed 24th March 2023].
- Settles, B. (2010), Active learning literature survey, Computer Sciences Technical Report 1648, University of Wisconsin–Madison. Available from: <https://burrsettles.com/pub/settles.activelearning.pdf> [Accessed 8th February 2023].
- Shannon, C. E. (1948), ‘A mathematical theory of communication’, *The Bell System Technical Journal* **27**, 379–423. Available from: doi:10.1002/j.1538-7305.1948.tb01338.x.
- Sokolova, M. & Lapalme, G. (2009), ‘A systematic analysis of performance measures for classification tasks’, *Information Processing & Management* **45**(4), 427–437. Available from: doi:doi.org/10.1016/j.ipm.2009.03.002.
- Tong, S. & Koller, D. (2001), ‘Support vector machine active learning with applications to text classification’, *Journal of machine learning research* **2**(Nov), 45–66.

A Data

A.1 Ten Most Common Tokens Students Questions Dataset

assertion	boldsymbol	$\cdot$	correct	$\frac{}{}$
left	mathbf	$\mathrm{}$	reason	right

A.2 English Stop-Words

a	about	above	after	again	against	ain
all	am	an	and	any	are	aren
aren't	as	at	be	because	been	before
being	below	between	both	but	by	can
couldn	couldn't	d	did	didn	didn't	do
does	doesn	doesn't	doing	don	don't	down
during	each	few	for	from	further	had
hadn	hadn't	has	hasn	hasn't	have	haven
haven't	having	he	her	here	hers	herself
him	himself	his	how	i	if	in
into	is	isn	isn't	it	it's	its
itself	just	ll	m	ma	me	mightn
mightn't	more	most	mustn	mustn't	my	myself
needn	needn't	no	nor	not	now	o
of	off	on	once	only	or	other
our	ours	ourselves	out	over	own	re
s	same	shan	shan't	she	she's	should
should've	shouldn	shouldn't	so	some	such	t
than	that	that'll	the	their	theirs	them
themselves	then	there	these	they	this	those
through	to	too	under	until	up	ve
very	was	wasn	wasn't	we	were	weren
weren't	what	when	where	which	while	who
whom	why	will	with	won	won't	wouldn
wouldn't	y	you	you'd	you'll	you're	you've
your	yours	yourself	yourselves			

B Code

B.1 Initial Labeled Set

```
def initial_train(model, X_train, y_train, size, X_test, rand):
    X_labeled, X_pool, y_labeled, y_pool = train_test_split(X_train,
                                                            y_train,
                                                            train_size=size,
                                                            random_state=rand)

    model.fit(X_labeled, y_labeled)
    y_pred = model.predict(X_pool)
    score_init = accuracy_score(y_pool, y_pred)
    return X_labeled, X_pool, y_labeled, y_pool, model, score_init
```

B.2 Query Strategies

```
#Random sampling
def RAND_function(iter, model, X_train, y_train,
                  size, X_test, y_test, rand):

    X_labeled, X_pool, y_labeled, y_pool, model, score_init
    =initial_train(model, X_train, y_train, size, X_test, rand)
    score_random_storer = [score_init]

    for i in range(iter):
        rows, columns = X_pool.shape
        idx = sample(range(rows), 1)
        X_labeled = vstack([X_labeled, X_pool[idx]])
        y_labeled = np.concatenate([y_labeled, y_pool.iloc[idx]])

        mask = np.ones(X_pool.shape[0], dtype=bool)
        mask[idx] = False
        X_pool = X_pool[mask]
        y_pool = y_pool.drop(y_pool.index[idx])
        model.fit(X_labeled, y_labeled)

        y_pred = model.predict(X_test)
        score_random = accuracy_score(y_test, y_pred)
        score_random_storer.append(score_random)
    return score_random_storer, X_labeled, y_labeled

# Least confident sampling
def LC_function(iter, model, X_train, y_train,
                size, X_test, y_test, rand):

    X_labeled, X_pool, y_labeled, y_pool, model, score_init
    =initial_train(model, X_train, y_train, size, X_test, rand)
    score_LC_storer = [score_init]

    for i in range(iter):
        y_probab = model.predict_proba(X_pool)
        confidences = 1-np.max(y_probab, axis=1)
```



```

least_certain_samples = np.argsort(confidences)[::-1][:1]

X_labeled = vstack([X_labeled, X_pool[least_certain_samples]])
y_labeled = np.concatenate([y_labeled,
                             y_pool.iloc[least_certain_samples]])
mask = np.ones(X_pool.shape[0], dtype=bool)
mask[least_certain_samples] = False

X_pool = X_pool[mask]
y_pool = y_pool.drop(y_pool.index[least_certain_samples])
model.fit(X_labeled, y_labeled)

y_pred = model.predict(X_test)
score_LC = accuracy_score(y_test, y_pred)
score_LC_storer.append(score_LC)
return score_LC_storer, X_labeled, y_labeled

# Margin Sampling
def MARG_function(iter, model, X_train, y_train,
                  size, X_test, y_test, rand):

    X_labeled, X_pool, y_labeled, y_pool, model, score_init
    =initial_train(model, X_train, y_train, size, X_test, rand)

    no_classes = len(np.unique(y_pool))-1
    score_MARG_storer = [score_init]

    for i in range(iter):
        y_probab = model.predict_proba(X_pool)
        arr = np.sort(y_probab, axis=1)
        diff = arr[:,no_classes] - arr[:,(no_classes-1)]
        idx = np.argmin(diff)
        idx = idx.flatten()
        X_labeled = vstack([X_labeled, X_pool[idx]])
        y_labeled = np.concatenate([y_labeled, y_pool.iloc[idx]])

        mask = np.ones(X_pool.shape[0], dtype=bool)
        mask[idx] = False
        X_pool = X_pool[mask]
        y_pool = y_pool.drop(y_pool.index[idx])
        model.fit(X_labeled, y_labeled)

        y_pred = model.predict(X_test)
        score_MARG = accuracy_score(y_test, y_pred)
        score_MARG_storer.append(score_MARG)
    return score_MARG_storer, X_labeled, y_labeled

# Entropy Sampling
def ENT_function(iter, model, X_train, y_train,
                 size, X_test, y_test, rand):

```

```

X_labeled, X_pool, y_labeled, y_pool, model, score_init
= initial_train(model, X_train, y_train,
                size, X_test, rand)
score_ENT_storer = [score_init]

for i in range(iter):
    y_probab = model.predict_proba(X_pool)
    entropy = np.sum(-y_probab[:,]*np.log2(y_probab[:,]), axis = 1)
    idx = np.argmax(entropy)
    idx = idx.flatten()
    # Adding the most uncertain instance to the labeled set
    X_labeled = vstack([X_labeled, X_pool[idx]])
    y_labeled = np.concatenate([y_labeled, y_pool.iloc[idx]])

    #Removing the chosen text instance from the pool
    mask = np.ones(X_pool.shape[0], dtype=bool)
    mask[idx] = False
    X_pool = X_pool[mask]
    y_pool = y_pool.drop(y_pool.index[idx])
    model.fit(X_labeled, y_labeled)

    y_pred = model.predict(X_test)
    score_ENT = accuracy_score(y_test, y_pred)
    score_ENT_storer.append(score_ENT)
return score_ENT_storer, X_labeled, y_labeled

```